COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 \| Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | Oct 1, 2026 | **DATE SUBMITTED:** | Oct 13, 2026 |
| **NAME:** | Ellaiza C. Vizcayno | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 10** **Graphs**

**1. PROCEDURE OUTPUT**

**Adjacency List**

**Code**

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head; // point new node to current head
        return newNode;
    }
    int N; // number of nodes in the graph
```

```
    public:
        adjNode **head; //adjacency list as array of pointers

        // Constructor
        DiaGraph(graphEdge edges[], int n, int N){

            // allocate new node
            head = new adjNode*[N]();
            this->N = N;

            // initialize head pointer for all vertices
            for (int i = 0; i < N; ++i)
                head[i] = nullptr;

            // construct directed graph by adding edges to it
            for (int i = 0; i < n; i++) {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;

                // insert in the beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

                // point head pointer to new node
                head[start_ver] = newNode;
            }
        }

        DiaGraph(const DiaGraph&) = delete;
        DiaGraph& operator=(const DiaGraph&) = delete;

        // Destructor
        ~DiaGraph() {
            for (int i = 0; i < N; i++) {
                adjNode* cur = head[i];
                while (cur != nullptr) {
                    adjNode* next = cur->next;
                    delete cur;
                    cur = next;
                }
            }
            delete[] head;
        }
};
```

```cpp
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val<< ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main(){

    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},
        {0,2,4},
        {1,4,3},
        {2,3,2},
        {3,1,4},
        {4,3,3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;

    for (int i = 0; i < N; i++){
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

**Output**

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

| Depth-First Search | |
|---|---|
| Code | |

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
            os << std::endl;
        }
    return os;
}
```

```cpp
template <typename T>
class Graph{
    public:
        // Initialize the graph with N vertices
        Graph(size_t N) : V(N){}

        // Return number of vertices in the graph
        auto vertices() const{
            return V;
        }

        // Return all edges in the graph
        auto &edges() const{
            return edge_list;
        }

        void add_edge(Edge<T> &&e){
            // Check if the source and destination vertices are within range
            if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
                edge_list.emplace_back(e);
            else
                std::cerr << "Vertex out of bounds" << std::endl;
        }
        // Returns all outgoing edges from vertex v
        auto outgoing_edges(size_t v) const{
            std::vector<Edge<T>> edges_from_v;
            for (auto &e : edge_list){
                if (e.src == v)
                    edges_from_v.emplace_back(e);
            }
            return edges_from_v;
        }

        // Overloads the << operator so a graph be written directly to a stream
        // Can be used as std::cout << obj << std::endl;
        template <typename U>
        friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

    private:
        size_t V; // Stores number of vertices in graph
        std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest){
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()){
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)){

                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()){
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

```cpp
template <typename T>
void test_DFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main(){

    using T = unsigned;
    test_DFS<T>();

    return 0;
}
```

**Output**

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

| Breadth-First Search | |
|---|---|
| Code | |

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (size_t i = 1; i <= G.vertices(); i++){
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph{
    public:
        Graph(size_t N) : V(N) {}
        auto vertices() const{
            return V;
        }

        auto &edges() const{
            return edge_list;
        }

        void add_edge(Edge<T> &&e){
            if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
                edge_list.emplace_back(e);
            else
                std::cerr << "Vertex out of bounds" << std::endl;
        }

        auto outgoing_edges(size_t v) const{
            std::vector<Edge<T>> edges_from_v;
            for (auto &e : edge_list){
                if (e.src == v){
                    edges_from_v.emplace_back(e);
                }
            }
            return edges_from_v;
        }

    private:
        size_t V;
        std::vector<Edge<T>> edge_list;
};


template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest){
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty()){
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

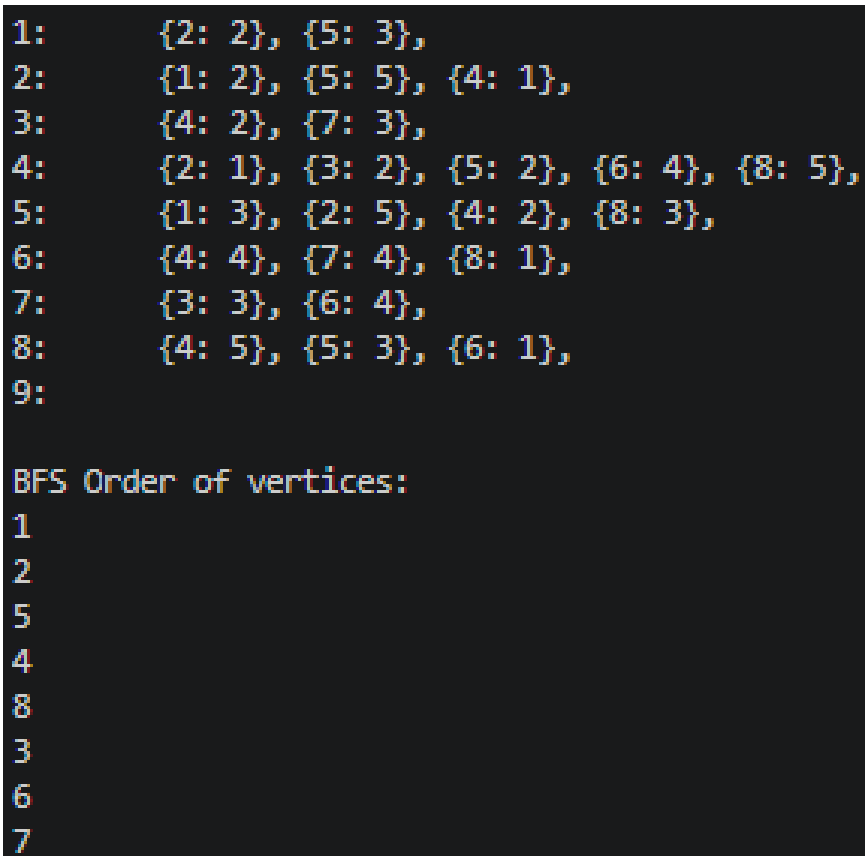
template <typename T>
void test_BFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<T>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main(){
    using T = unsigned;
    test_BFS<T>();

    return 0;
}
```

| | |
|---|---|
| **Output** |  |
| **Observations** | In breadth-first search, it seems to use the data structure of a queue. Where it pushes the outgoing vertices of the vertex being looked at into a queue. This queue is used to check if the current vertex is already visited or not. If the vertex has already been visited, then this vertex is ignored. If the vertex hasn't been visited, it is pushed into a list that contains the order of vertices visited. Also, the outgoing vertices of that vertex will be pushed onto a queue that contains the list of vertices that will be checked one by one. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

| | |
|---|---|
| **1** | A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertices from the same vertex. Discuss which algorithm would be most helpful to accomplish this task. |
| **Answer :** | When it comes to starting from one location until it finishes, and then backtracking to explore other vertices from the same vertex, the algorithm most helpful for this task is the **Depth-First Search Algorithm.** This is because the person already described the definition of the said algorithm. Where it finishes a path from start until end, then it goes back to search for other paths until they finish and it repeats until all paths have been discovered or visited. |

| | |
|---|---|
| **2** | Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation |
| **Answer :** | Depth-First Search reminded me of pre-order tree traversal. Pre-order traversal uses root->left->right. This means that it will continue traversing the left side of the tree until it reaches a leaf node, then it moves to the right nodes of the root after backtracking. This description is similar to DFS, as they visit a path until it ends then backtracks to find other paths until they finish. |

| | |
|---|---|
| **3** | In the performed code, what data structure is used to implement the Breadth First Search? |
| **Answer :** | In implementing the Breadth-First Search, I noticed that it uses the queue data structure. It uses it when looking at the current vertex and the queue of vertices. It checks whether the current vertex has already been visited, and it also pushes the outgoing vertices of the unvisited vertex. |

| | |
|---|---|
| **4** | How many times can a node be visited in the BFS? |
| **Answer :** | Breadth-First Search only allows one visitation of each node. While it can be pushed onto the queue multiple times to be checked, it only gets added to the visit order once, when it's first taken off the queue and found not to be in the visited set. This avoids infinite loops in graphs with cycles because the visited set catches any vertex that was already processed, so it gets skipped instead of expanded again. |

## Conclusion

In this activity, I learned two ways in traversing graphs and general trees. These two ways are **Depth-First Search** and **Breadth-First Search**. DFS can be compared to pre-order tree traversal as they both finish one path first, starting from the root, until it ends, then backtracking to find other paths until they as well end. In other words, they go deep in and come back once they reach a dead end and look for other paths. On the other hand, BFS in trees goes level by level, but is a bit different when it comes to graphs as they can have multiple cycles, although they follow the same logic. They first check whether the node or vertex has been visited. If not, they push it into another list where it contains the order of which the vertices are visited, and they push the outgoing vertices or children onto the queue where they check if they have been visited. If they have been visited, then they will just be ignored.

At first, I was a bit intimidated by the sample code. Although, once you study it for hours, you eventually understand the concept. Actually, the concept is easy to understand and visualize. You just have to understand the process behind each search or traversal, because once you get the concept, everything comes with ease. Overall, I might still have a hard time with the code but I get the concept and idea of each traversal.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **N/A**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **N/A**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **N/A**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.